

Proyecto II

Tecnológico de Costa Rica
Escuela de Ingeniería en Computación
Bases de Datos II (IC 4302)
Segundo Semestre 2025



1. Objetivo General

- Desarrollar un prototipo de aplicación con bases de datos SQL y NoSQL.

2. Objetivos Específicos

- Implementar un Selenium Web Scraper utilizando Python.
- Desarrollar scripts de procesamiento de datos mediante Spark SQL.
- Implementar un sistema de búsqueda de productos.
- Automatizar la solución utilizando Docker, Kubernetes y Helm Charts.
- Desarrollar arquitecturas de microservicios en Kubernetes.
- Implementar servicios básicos de extracción de entidades.
- Instalar y configurar servicios de bases de datos NoSQL.
- Diseñar e implementar una aplicación web utilizando inteligencia artificial generativa.

3. Datos Generales

- Valor del proyecto: 20%
- Nombre del proyecto: Products Search V2
- Trabajo en equipo: grupos de máximo 5 personas.
- Fecha de entrega: viernes 07 de noviembre de 2025, antes de las 11:59 p. m.
- Cualquier indicio de copia se calificará con nota 0 y se procederá según el reglamento. La copia incluye código/configuraciones encontrados en Internet y utilizados parcial o totalmente sin el debido reconocimiento al autor.
- La revisión se realizará de forma virtual, mediante citas en las que todas las personas integrantes del grupo deben estar presentes. El proyecto debe estar completamente automatizado y el/la profesor(a) decide en qué computadora probar.
- En la documentación se deben incluir instrucciones claras para ejecutar el proyecto.
- Se deben incluir pruebas unitarias para verificar los componentes individuales. Si no están presentes, la calificación será 0.
- Se espera que todas las personas integrantes del grupo entiendan la implementación entregada.
- Se espera la aplicación de buenas prácticas de programación (cuando apliquen):
 - ◆ Documentación interna y externa.
 - ◆ Estándares de código.
 - ◆ Diagramas de arquitectura.
 - ◆ Diagramas de flujo.
 - ◆ Pruebas unitarias.

- #### 4. Descripción

Web Scraper

Esta es una aplicación escrita en Python que se ejecuta fuera de Kubernetes. Debe extraer información de productos de un sitio de comercio electrónico. El sitio seleccionado debe exponer descripciones de productos, títulos, características y comentarios. Se debe guardar un archivo HTML por producto. Deben descargarse 500 productos o más para que el proyecto tenga sentido, estos deberán subirse al bucket de S3 **ic-tec-dataset**, en una carpeta propia de cada grupo, para su posterior procesamiento. Se recomienda:

- Descargar tantos productos como sea posible de forma local (en una carpeta de sus computadoras).
- Cuando tengan una cantidad significativa de archivos, súbanlos al bucket de S3 mediante un comando como: `aws s3 sync {CARPETA_LOCAL} s3://ic-tec-dataset/{CARPETA_GRUPO}/`

Para esto, pueden instalar AWS CLI siguiendo:

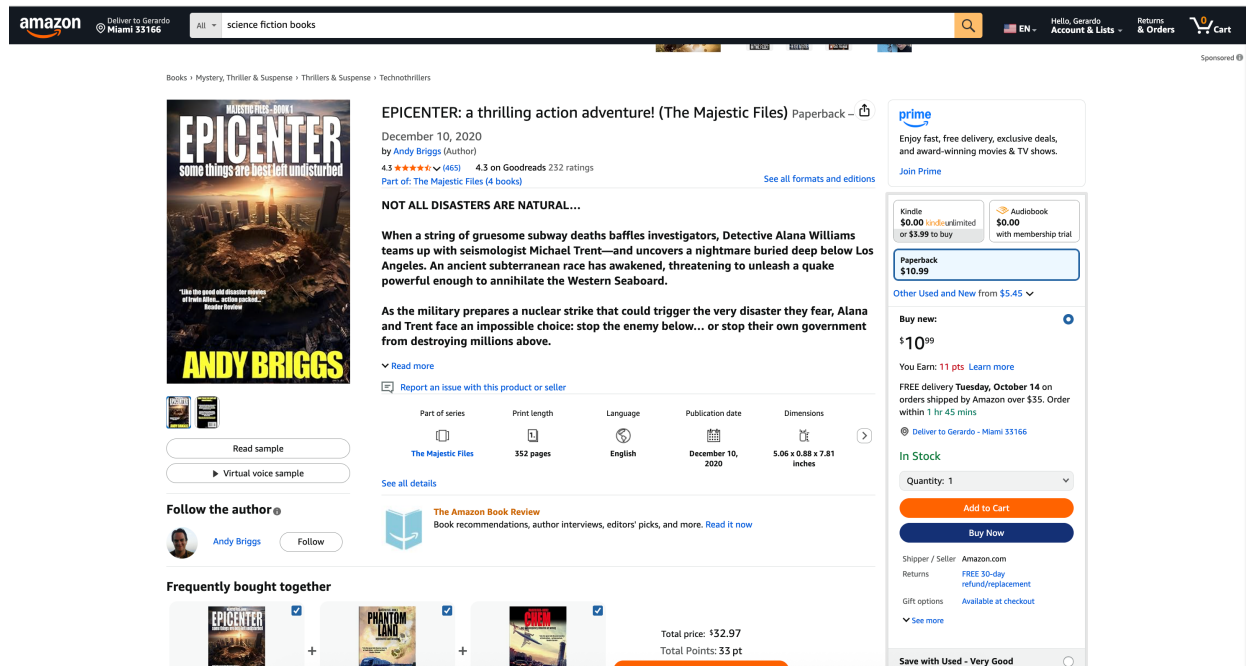
<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html> Y consultar la documentación del comando sync aquí:

<https://docs.aws.amazon.com/cli/latest/reference/s3/sync.html>

Por ejemplo Amazon, si queremos obtener libros de ciencia ficción, el web Scraper deberá buscar los libros:

The screenshot shows the Amazon website interface. At the top, there's a navigation bar with the Amazon logo, delivery location (Miami 33166), and a search bar. Below the navigation bar, there's a banner for "science fiction books" with 1-48 of over 100,000 results. The main content area displays a grid of book covers and titles. The books shown include "The Worst Ship in the Fleet", "The Worst Spies in the Sector", "The Worst Pirate Hunters in the Fringe", "EPICENTER", "Night Owls", "Federation Chronicles", "Backyard Starship", and "Hive". Each book entry includes the title, author, and a brief description. The sidebar on the left contains filters for book format (Kindle Edition, Paperback, Audible Audiobook, Hardcover, Large Print, Loose Leaf, Audio CD, Board Book, Spiral-bound), customer reviews (4 stars & up), deals & discounts, and book language (English, Spanish, French, German, Italian, Russian, Chinese (Simplified)).

Ingresar a cada uno de ellos:



Y extraer su información, esta información se obtiene en formato HTML, el resultado debe ser almacenado en el bucket. Se debe hacer esto para cada item y cuando se hayan procesado todos en una página, se debe pasar a la siguiente página.

Controller

El componente Controller, que se ejecuta como un CronJob de Kubernetes, se ejecutará de forma periódica. Su trabajo consiste en:

- Listar todos los archivos en la carpeta `s3://ic-tec-dataset/{CARPETA_GRUPO}/`.
- Crear un documento por cada archivo en una colección llamada "ingestion", este documento deberá guardar información como: nombre del archivo, tamaño en bytes, hash MD5 y estado (inicialmente **new**).
- Si ya existía un documento para un archivo específico, se deberá comparar su MD5, si es igual, se ignora, si es diferente, se actualiza la información y el estado se establece como **modified**.
- Publicar un mensaje en una cola de RabbitMQ para su procesamiento posterior.

Beautiful Soup Parser

Este componente se ejecuta como un Deployment de Kubernetes con un mínimo de dos réplicas. Su trabajo consiste en:

- Leer los mensajes publicados por el componente Controller.
- Actualizar la colección "ingestion", estableciendo que el procesamiento del archivo ha iniciado.

- Descargar el archivo HTML desde el bucket de S3. Analizar (parsear) el archivo utilizando la biblioteca Beautiful Soup para extraer toda la información relevante del producto.
- Almacenar la información del producto como un archivo JSON en una carpeta llamada raw en el disco compartido entre los pods de la solución, este disco debe ser de tipo ReadWriteMany.
- Actualizar la colección “ingestion”, estableciendo que el procesamiento del archivo se ha completado.
- Publicar un mensaje en una cola de RabbitMQ para la extracción de entidades.

Spacy Entity Extractor

Este componente se ejecuta como un Deployment de Kubernetes con un mínimo de dos réplicas. Su trabajo consiste en:

- Leer los mensajes publicados por el componente Beautiful Soup Parse.
- Actualizar la colección “ingestion”, estableciendo que la extracción de entidades del archivo ha iniciado.
- Utilizar la funcionalidad por defecto de spaCy para realizar Named Entity Recognition (NER). Estas entidades se almacenarán en un nuevo campo de tipo array llamado entities.
- Guardar el archivo resultante en una carpeta llamada augmented dentro del disco compartido entre los pods.
- Actualizar la colección “ingestion”, estableciendo que la extracción de entidades del archivo se ha completado.

Spark Processor Job

Este componente se ejecuta como un CronJob de Kubernetes y se ejecutará de forma periódica. Su trabajo consiste en:

- Leer la carpeta augmented y normalizar los datos de la siguiente forma:
 - Asegurar que todos los textos comiencen con mayúscula (incluyendo las entidades).
 - Garantizar que todas las fechas tengan el formato DD/MM/YYYY.
 - Crear una descripción de no más de 140 caracteres (la implementación es libre, cada grupo puede definir su enfoque).
 - Utilizar las entidades reconocidas para relacionar productos. Por ejemplo, si en un artículo se reconoce la entidad “madera”, agregar una sección al documento con un máximo de 10 productos relacionados en los que también se reconoció la entidad “madera”.
- Una vez transformado, publicar el resultado en una colección llamada documents en MongoDB Atlas.

Configuración de Atlas Search

Mediante la utilización de Atlas Search (<https://www.mongodb.com/products/platform/atlas-search>), se deberá crear un índice de búsqueda sobre la colección llamada documents. Importante:

- Definir un mapping adecuado para los datos procesados (<https://www.mongodb.com/docs/atlas/atlas-search/define-field-mappings/>).
- Asegurar que se definan facets para todas las características y entidades identificadas (<https://www.mongodb.com/docs/atlas/atlas-search/facet/>).
- Verificar que se pueda realizar highlighting de los resultados de búsqueda (<https://www.mongodb.com/docs/atlas/atlas-search/highlighting/>).

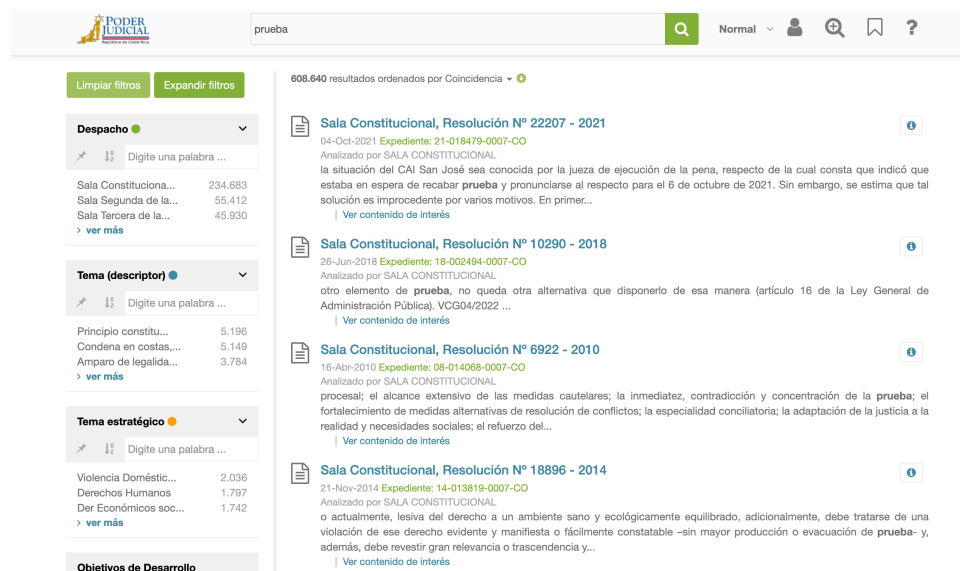
RestAPI (Node.js)

Se debe implementar una API REST en Node.js que se ejecute en Vercel. Esta API deberá exponer los endpoints requeridos por la interfaz gráfica e interactuar con MongoDB Atlas Search y Firestore. Además, deberá contar con un mecanismo de seguridad, cada grupo deberá investigar, implementar y documentar el mecanismo elegido.

UI

Usando Vercel, React, Vite, Firestore y TailwindCSS, se deberá implementar una interfaz gráfica que permita:

- Registrarse Iniciar sesión
- Cerrar sesión
- Buscar y filtrar productos
- El diseño de la interfaz queda a criterio de cada grupo. Como referencia, puede considerarse la implementación del Poder Judicial de Costa Rica: <https://nexuspj.poder-judicial.go.cr/>



- Al dar clic sobre alguno de los facets, se deberían de filtrar los documentos resultantes, al dar clic sobre alguno de los resultados se deberá mostrar toda la información del artículo.

Si el grupo así lo decide, puede utilizar algún sistema de inteligencia artificial para generar únicamente la interfaz gráfica, no obstante, se debe agregar a la documentación una sección que explique el proceso, los prompts utilizados y una opinión grupal sobre el uso de estas herramientas.

Automatización

Con excepción de Selenium Web Scraper, todos los componentes de la aplicación deben estar debidamente automatizados mediante Vercel o Helm Charts.

Documentación

Se deberá entregar una documentación que, como mínimo, incluya:

- Instrucciones claras sobre cómo ejecutar el proyecto.
- Pruebas realizadas, con pasos para reproducirlas.
- Resultados de las pruebas unitarias.
- Recomendaciones y conclusiones (al menos 10 de cada una).
- Cobertura de todos los componentes implementados o instalados/configurados. En caso de que algún componente no esté implementado, no podrá documentarse y esto impactará la completitud de la documentación.
- Referencias bibliográficas donde aplique.
- Tabla que resuma el estado de las funcionalidades al momento de la entrega (que está implementado, que no está implementado, que problemas tiene el trabajo presentado).

Imaginen que la documentación está dirigida a una persona con conocimientos básicos en computación, y que se espera que pueda ejecutar y probar el proyecto sin inconvenientes. El uso de imágenes, diagramas, fragmentos de código (code snippets), videos, etc., puede ser de gran ayuda. La documentación debe estar escrita en Markdown y exportada a PDF.

5. Recomendaciones

- Utilizar telnet/curl/[postman](#) para interactuar con los componentes y verificar que los mismos están funcionando correctamente.
- Utilizar [Docker Desktop](#) para instalar Kubernetes en Windows.
- Utilizar [Minikube](#) para ejecutar Kubernetes en Linux
- Realicen peer-review/checkpoints semanales y no esperen hasta el último momento para integrar su aplicación.
- Crear un repositorio de GitHub e invitar al profesor, pueden usar el repositorio de alguno de los integrantes del grupo.
- Realizar commits y push regulares.

6. Entregables

- Documentación.
- Docker files, Docker Compose files y Helm Charts junto con todos los archivos/scripts requeridos para ejecutar su proyecto.

7. Evaluación

Funcionalidad / Requerimiento	Porcentaje
Documentación	10%
Implementación (*): ● Controller (5%) ● Web Scraper (15%) ● BeautifulSoup(15%) ● Spacy Entity Extractor (10%) ● Spark Processor Job (15%) ● Configuración Firestore, Mongo Atlas Search y RabbitMQ (10%) ● UI y Rest API (20%)	90%
	100%

(*) Todo tiene que estar debidamente automatizado, de lo contrario se calificará con una nota de 0.